The 50th IEEE Conference on Local Computer Networks (LCN)
October 14-16, 2025, Sydney, Australia

EFCC:
Ethernet Frame Crafter & Capture for TSN Research

Akram BEN AHMED, Takahiro HIROFUCHI, Takaaki FUKAI
Intelligent Platform Research Institute (IPRI)
The National Institute of Advanced Industrial Science and Technology (AIST)
Tokyo, Japan

This work is based on results obtained from
"Research and Development Project of the Enhanced Infrastructures for Post-5G Information and Communication Systems"(JPNP20017),
commissioned by the New Energy and Industrial Technology Development Organization (NEDO)

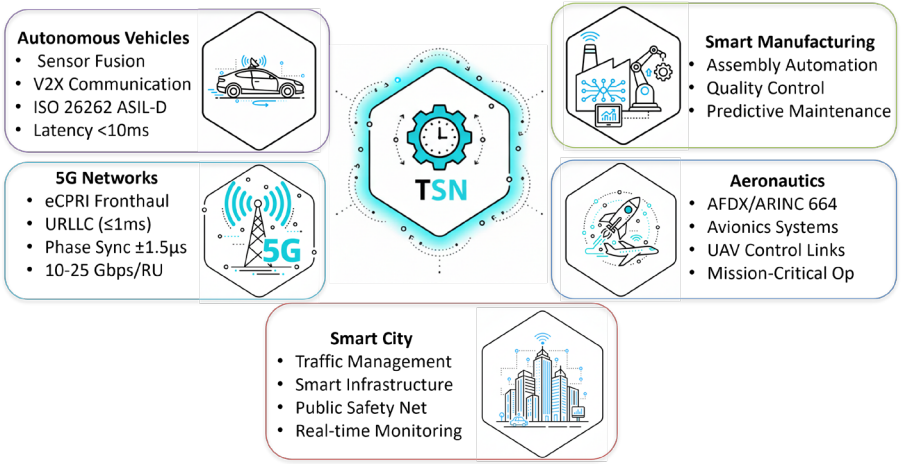
Outline

- TSN opportunities and challenges
- Motivation and solution
- EFCC architecture and implementation
- Evaluation environment and results
- Conclusion and Future work

Time Sensitive Networks:
Enabling Deterministic Communication Across Industries

- Emerging applications are latency-sensitive.
- Need for guaranteed ultra-low latency and minimal jitter (µs or ns scale).

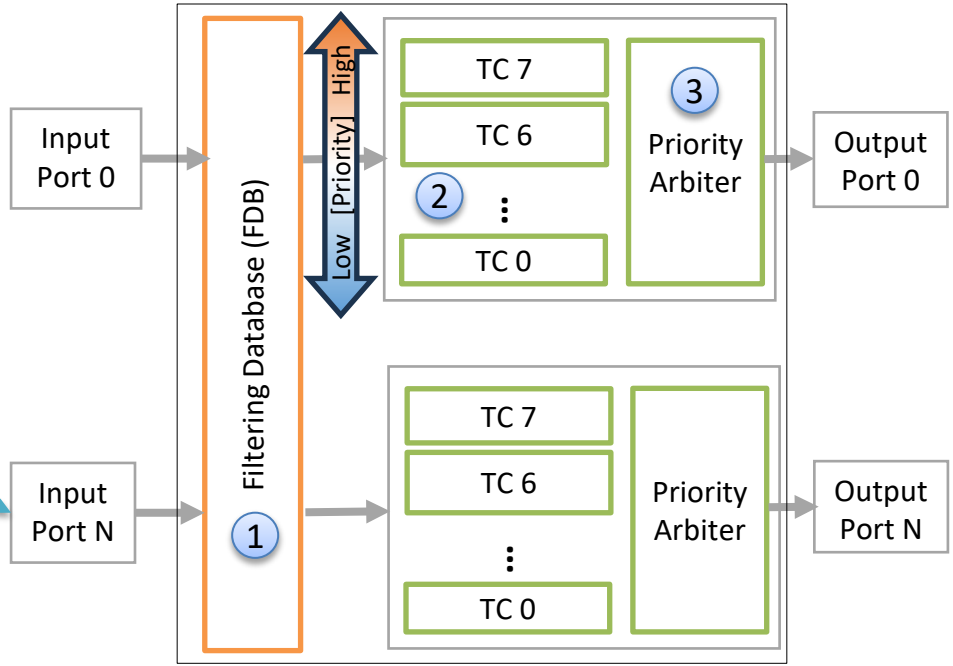
- **Time-Sensitive Network (TSN)** is a set of IEEE 802.1 standards that enhance standard Ethernet networks to provide **deterministic, ultra-reliable, and low-latency communication.**



Feature	Benefit	Impact
Deterministic Latency	Guarantees critical data packets arrive exactly on time (very low jitter)	Essential for **Smart Grids** and **Smart Manufacturing**
Microsecond Synchronization	Ensures all connected devices operate on a single, shared, highly precise clock	Critical for **Autonomous Vehicles** and measurements in **Aeronautics**
Traffic Scheduling	Reserves fixed time slots for time-critical data, preventing network congestion	Guarantees operational integrity and safety for highly critical tasks
Seamless Network Convergence	Allows critical (OT) and non-critical (IT) data to share one standard Ethernet network	Reduces infrastructure and maintenance costs in **Smart Factories**
High Reliability & Redundancy	Provides seamless frame duplication over multiple paths (fault tolerance).	Zero packet loss in **Aerospace** and **Smart Grids** and **Manufacturing**
Open Standard Interoperability	Standardizes real-time communication protocols across the entire network layer.	Enabling multi-vendor systems in **Smart Cities**

How does a TSN switch work?

- TSN controls the frame transmission interval for each flow at the source host and intermediate switches according to a predefined algorithm
- Each flow is identified by a VLAN ID, MAC address, etc..
- The priority of each flow is determined by the priority field (PCP) in the VLAN header.
- Each transmit port on TSN-enabled communication devices has up to eight Traffic Classes (TCs) corresponding to priority levels.
- Each TC is a queuing system; each assigned a transmission algorithms (CBS, ATS, etc.).

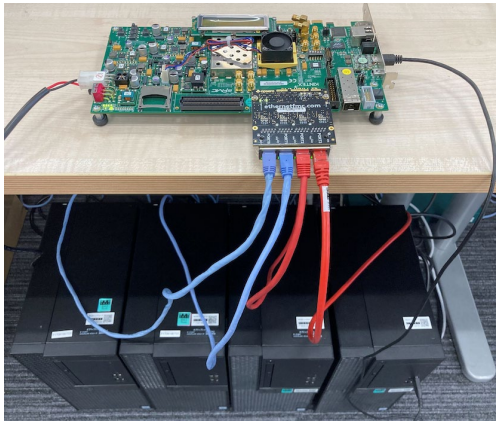


The high-level overview of a TSN switch

Mechanism:

1. *Identify the sending port and enqueue the frame in the TC queue matching its priority.*
2. *Check if the head-of-queue frame is eligible for transmission per traffic shaping algorithm.*
3. *The priority arbiter sends higher-TC frames first; lower-TC frames transmit only if higher-TC queues are empty.*

What we have achieved so far: The AIST-TSN Project



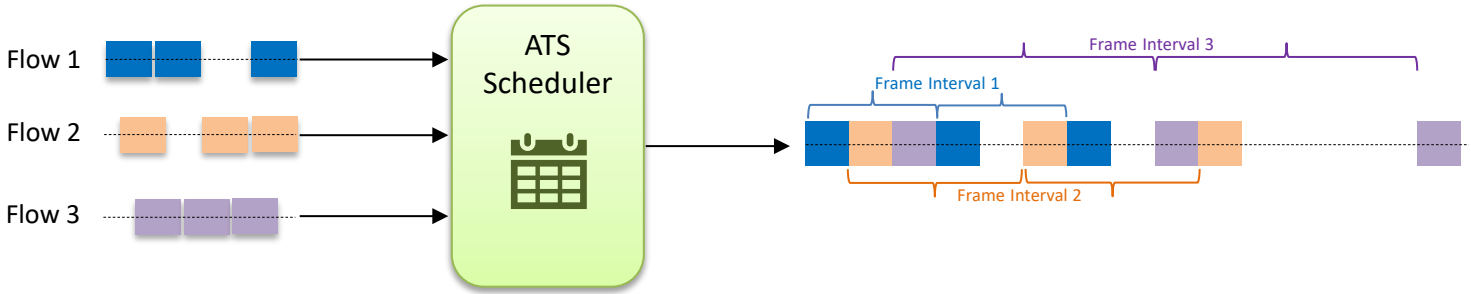
- Most of the existing works evaluate TSN functionalities by simulations or using commercial off-the-shelf hardware
  - Hard to evaluate on real (physical) environment with fine-grained analysis
  - Lack of unified evaluation hardware platform to compare the methods
- AIST-TSN is an **open-source** project offering **hardware design support** to accelerate TSN adoption for developers, researchers, and system integrators.
  - 3 M$ assigned for this project for 5 years
- Hardware design of a reconfigurable architecture of **an FPGA-based L2 network switch** supporting TSN.
- It supports two different transmission algorithms:
  - L2 switch supporting Credit Based Shaper (CBS) [1]
  - L2 switch supporting Asynchronous Traffic Shaper (ATS) [2]

[1] FPGA-based Network Switch Architecture Supporting Credit Based Shaper for Time Sensitive Networks, ETFA2024
[2] Hardware design and Evaluation of an FPGA-based Network Switch Supporting ATS for TSN, IEEE Access 2024

Asynchronous Traffic Shaper (ATS)

Asynchronous Traffic Shaper (ATS)

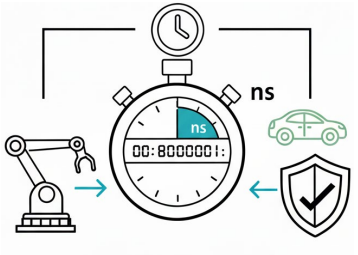
- **Mechanism:** Eligibility time system where a frame is transmitted at its calculated time.
- **Primary Goal:** Latency and jitter guarantees to ensure that the Worst-case End-to-End latency is deterministic and minimized
- **Multi-Flow per TC**: Each individual flow within a TC is assigned its own shaper and Eligibility time, providing scalable control.
- **Burst transfer:** Tolerated within a specified range
- **Complexity:** Moderate complexity (ATS is carefully designed to mitigate the implementation complexity)



Accurate TSN measurement is not easy!

Requirements:

- To build and deploy reliable TSN systems, we must rigorously test timing constraints:
  - Find the performance limits of Proof-of-Concept devices.
  - Ensure minimal variations within few nanoseconds
  - Troubleshoot complex, transient timing failures.
  - Validate new TSN mechanisms (e.g., shapers, schedulers).

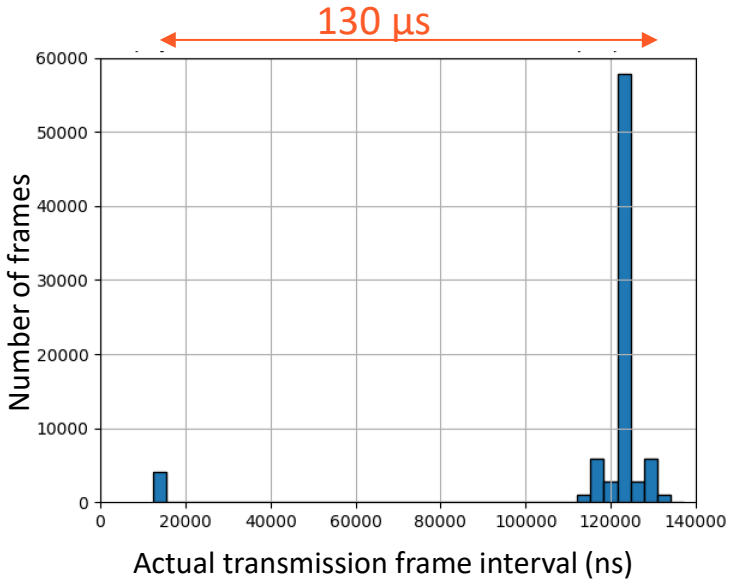


Existing solutions problems:

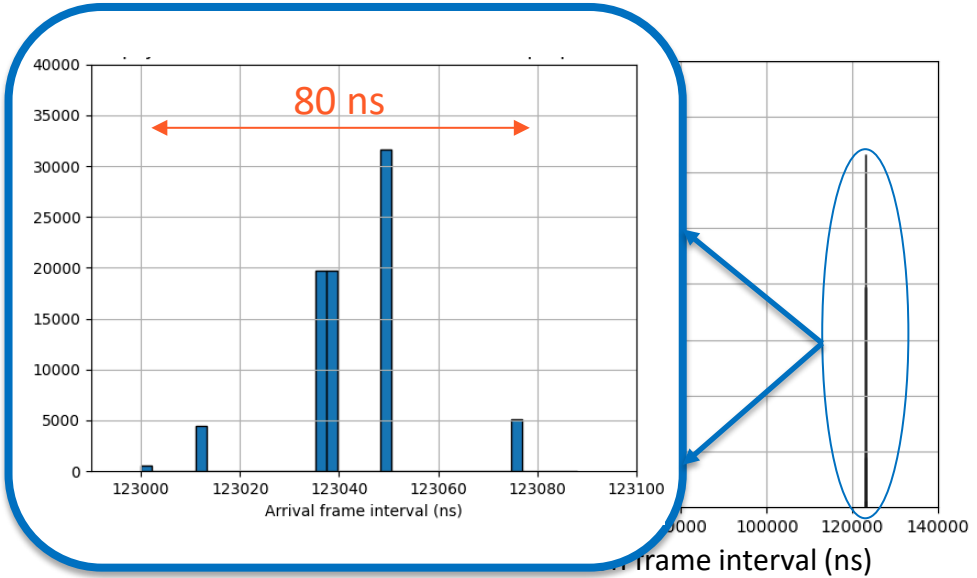
- Software Tools: OS/kernel **high jitter**
  - No precise control over frame interval.
  - Cannot simulate deterministic traffic.
  - Jitter in µs range
- Missing Capability: The **inability to mix multiple flows** with precise timing.
  - This is essential for real-world TSN testing.
- Commercial Hardware Tools: Expensive, **closed-source**, and inflexible.
  - Cannot be easily adapted for novel research (e.g., custom protocols).

TSN Measurement: The precision problem

- Conduct packet transmission experiment between two host machines equipped with Intel i210 NIC
  - Hardware offload option
    - Disabled : Packet transmission times are enforced by Linux software
    - Enabled : Packet transmission times are enforced by the NIC's LaunchTime hardware clock
- TX_Host performs 10 seconds transmission (81274 frames) with a frame interval of 123040ns
- Frame Interval was observed at the RX_Host machines



Hardware offload disabled



Hardware offload enabled

TSN Measurement: The target requirements

- 5G / Telecom Fronthaul (eCPRI/CPRI over TSN)
  - IEEE 802.1CM / ITU-T G.8273.2 Class C/D [Ref. 1]
  - Baseband I/Q transport and antenna phase alignment require ultra-low jitter to maintain coherent MIMO timing (**~10 ns**)
- Industrial Motion Control / Robotics
  - IEEE 802.1AS-rev, IEC/IEEE 60802 TSN Profile [Ref. 3]
  - Multi-axis drives and real-time servo loops require sub-µs latency and ns-scale jitter for deterministic control (**< 30-50 ns**)
- Power Grid Protection & Phasor Measurement
  - IEC 61850-9-3, IEEE 1588 Power Profile [Ref. 5]
  - Time alignment of sampled values and GOOSE events ensures precise fault isolation and protection switching (**≤ 80 ns**)
- Automotive TSN / ADAS sensor fusion
  - IEEE 802.1AS, 802.1Qbv/Qav, IEEE 1722 [Ref. 2]
  - Sensor fusion, radar, and camera timestamp alignment depend on few ns synchronization precision (**≤ 50-80 ns**)
- High-end Test & Measurement
  - IEEE 1588 High-Accuracy Profiles, NIST/ITU studies [Ref. 4]
  - Distributed oscilloscopes and radar arrays require ns-level timestamping accuracy for correlation (**< 10-50 ns**)

[Ref. 1] https://calnexsolutions.atlassian.net/wiki/spaces/GDW/pages/10027019/ITU-T+G.8273.2+T-BC+and+T-TSC+One-Page+Summary
[Ref. 2] https://1.ieee802.org/tsn/iec-ieee-60802/
[Ref. 3] https://standards.ieee.org/ieee/61850-9-3/6167/
[Ref. 4] https://1.ieee802.org/maintenance/802-1as-2020-cor1-corrigendum-to-ieee-standard-802-1as-2020/
[Ref. 5] https://www.nist.gov/el/intelligent-systems-division-73500/introduction-ieee-1588

Our motivation summary

- Shift in Focus: TSN Research has moved steadily from theoretical modelization and simulation toward building and testing real working systems.
- Practical Requirement: Real-world TSN systems must accurately transfer (few ns variation) a mixture of traffic flows, each with different characteristics
  - Frame sizes, frame intervals, burst sizes
- Crucial Need: These requirements make a **fully customizable and reconfigurable network measurement environment** more needed than ever for both academic and industrial research and development.
  - Three technical challenges:
    1. How to generate frames with accurate timing?
    2. How to record accurate timestamp on transmitting and receiving frames?
    3. How to allow user to configure flows?

Our solution:
A Customizable FPGA-Based Approach

AIST
Create the Future, Collaborate Together

Goal:

Developing a practical open-source FPGA-based measurement framework for TSN research and development.

Contributions:

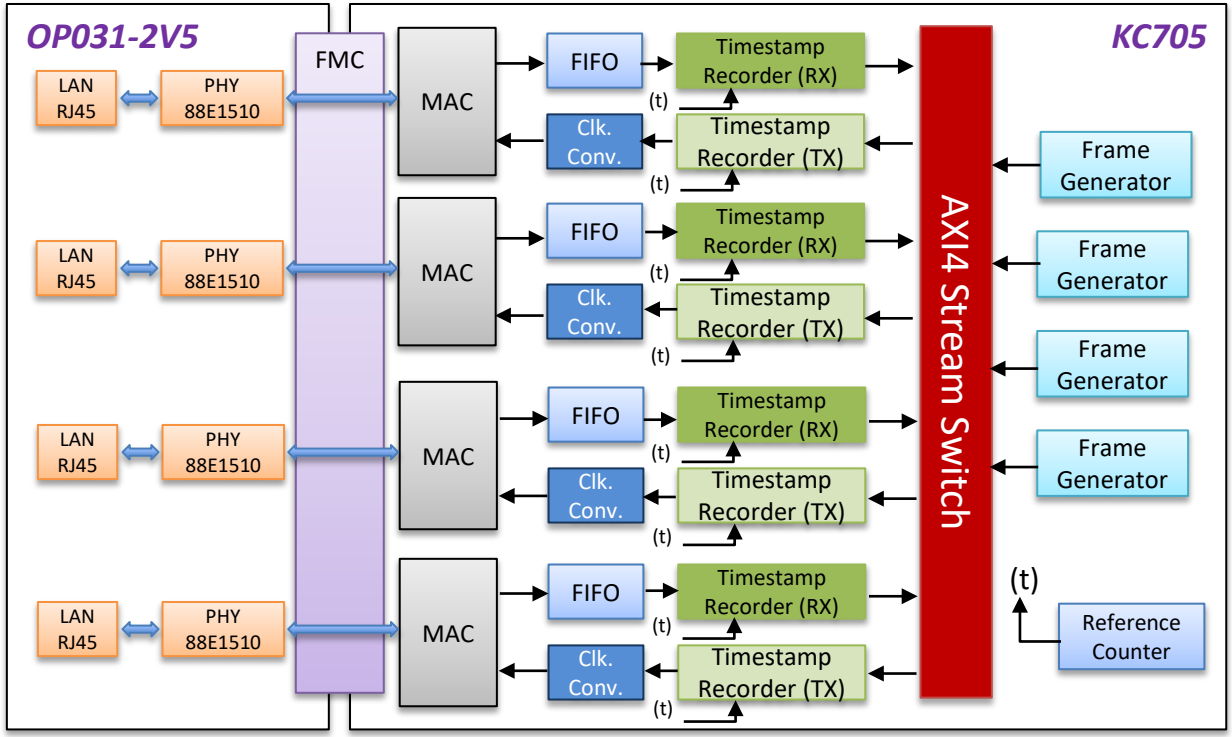
1. An FPGA-based ***Ethernet Frame Crafter & Capture*** (EFCC) for precise frame interval measurement and burst control
   - Hardware-level control bypasses OS jitter, achieving ns-level precision
2. A flexible, concurrent system supporting the mixing of multiple flows with independent characteristics.
   - Independent Frame Generation and Capture functions at 1GbE line rate.
3. An open-source and highly portable design for the research community.
   - Full flexibility for TSN research
   - https://github.com/CCIRT/aist-tsn-efcc

Outline

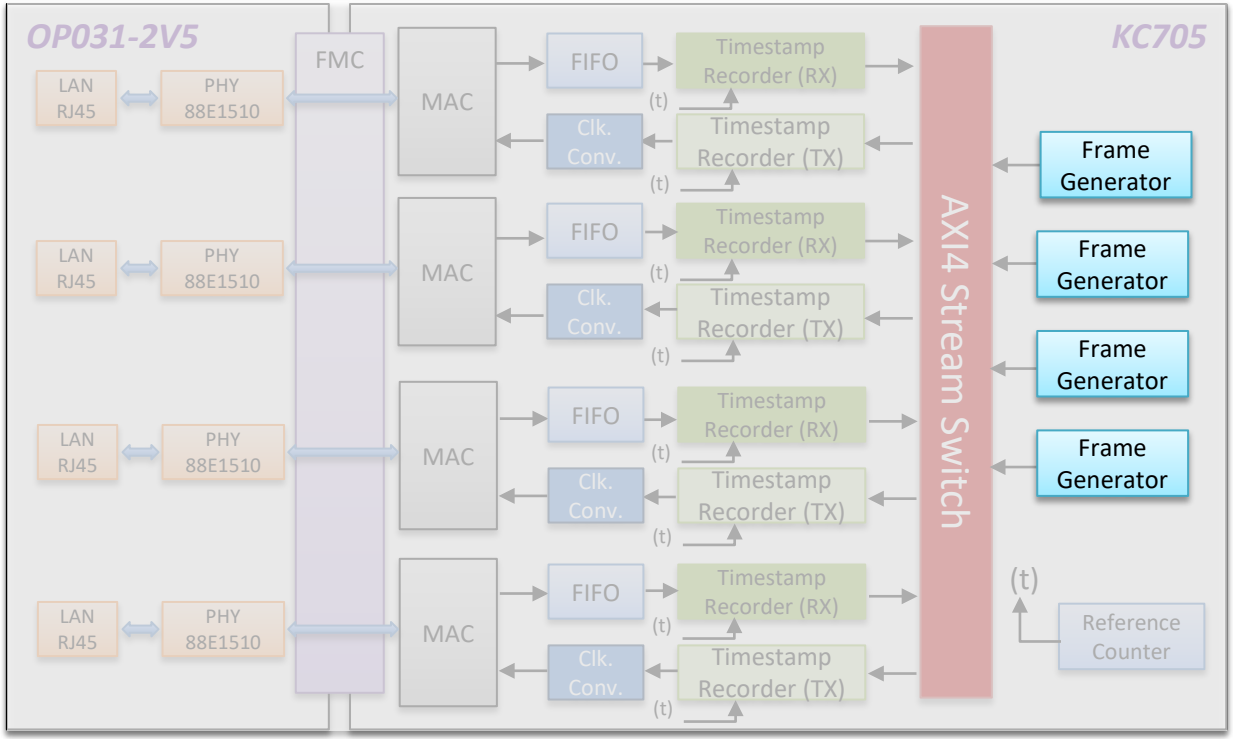
- TSN opportunities and challenges
- Motivation and solution
- EFCC architecture and implementation
- Evaluation environment and results
- Conclusion and Future work

EFCC Architecture

- 1000BASE-T at 1 Gbps Maximum designed speed
- Frame size can be set from 64 bytes to:
  - 1518 bytes (without VLAN TAG)
  - 1522 bytes (with VLAN TAG)
- Support UDP or RAW IPv4 frames
- Prototyped with an affordable FPGA board
  - AMD Kintex 7 FPGA KC705 Evaluation Kit plus Opsero OP031-1V8 Ethernet FMC
  - Portable to other commodity FPGA boards

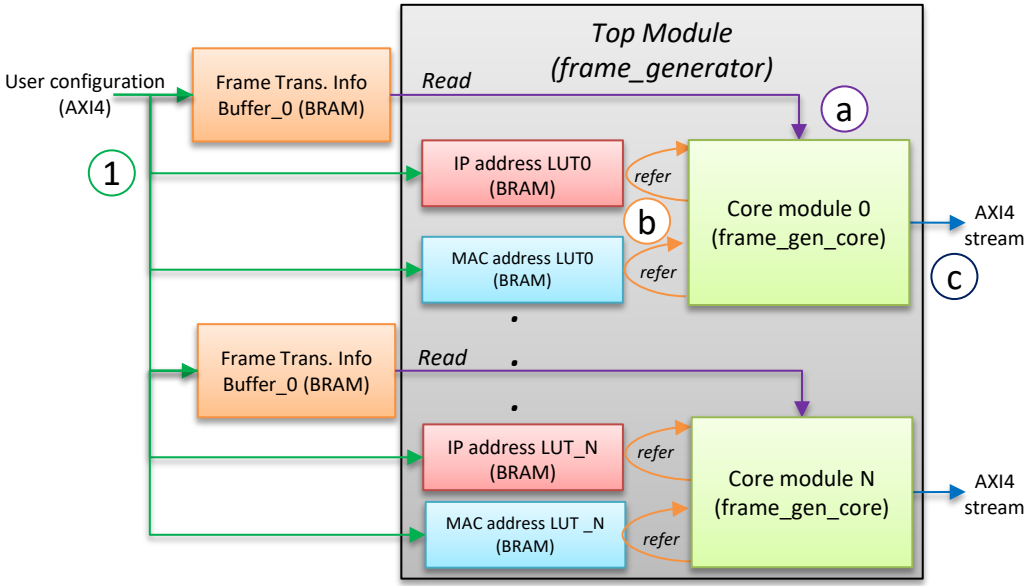


EFCC Architecture: Frame Generator



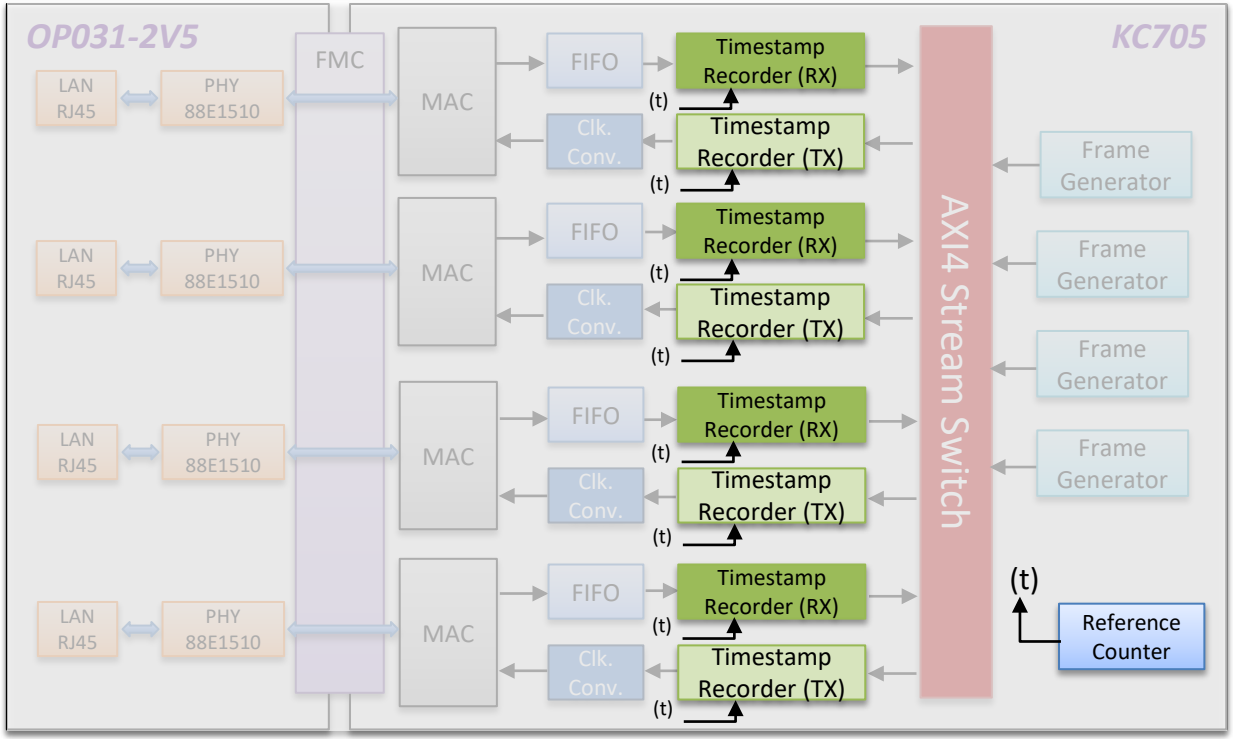
Frame Generator: Architecture

- The number of *Core Modules* is configurable to the number of physical output ports
- Two BRAM blocks containing:
  - IP addresses
  - MAC addresses
- *Frame Transmission Information Buffer* stores the information of the frame sequence to be transmitted:
  - ***Frame* entry**
    - An entry for the target frame to be sent
  - ***NOP* entry**
    - An entry to adjust frame intervals
  - ***EOL* entry**
    - An entry to indicate the end of a frame sequence
- Configurable Frame Interval:
  - Incrementable by 1 cycle @125MHz (8ns)



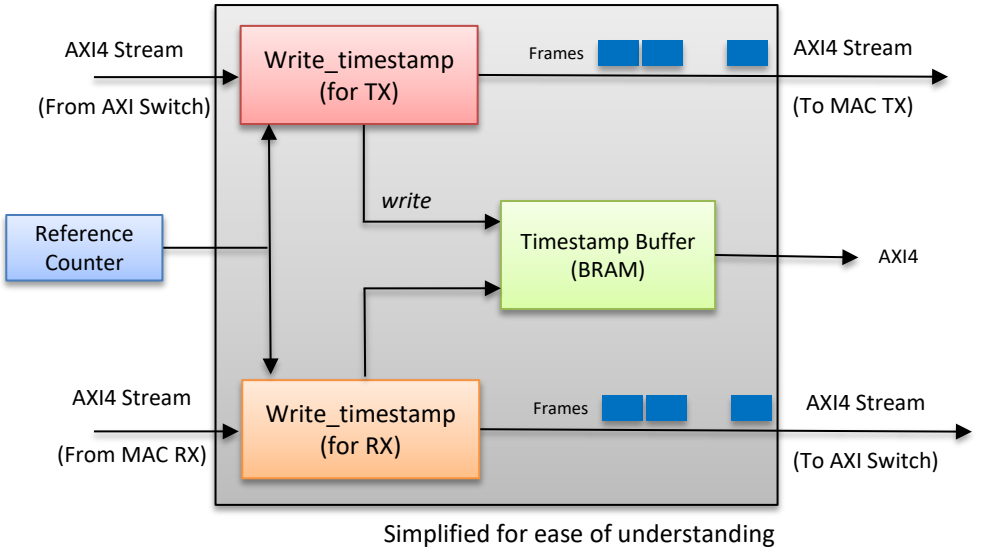
1. Users first setup configuration buffers
2. Core module:
   a. Reads configured frame sequence
   b. Refers to address buffers
   c. Generate frames and send them AXI4 Stream Switch

EFCC Architecture: Timestamp Recorder



Timestamp Recorder: Architecture

- The “write timestamp” module extracts the counter value of the time at which the beginning of AXI4-Stream passed through.
- The extracted value counter is then stored in the “Timestamp Buffer”
- The counter value is read from the “Ref. Counter” module
  - unsigned integer 32-bit counter



Outline

- TSN opportunities and challenges
- Motivation and solution
- EFCC architecture and implementation
- Evaluation environment and results
  - Timing accuracy
  - Frame capture capability
  - Frame generation capability
  - Latency bound of multiple flows
  - Hardware complexity
- Conclusion and Future work

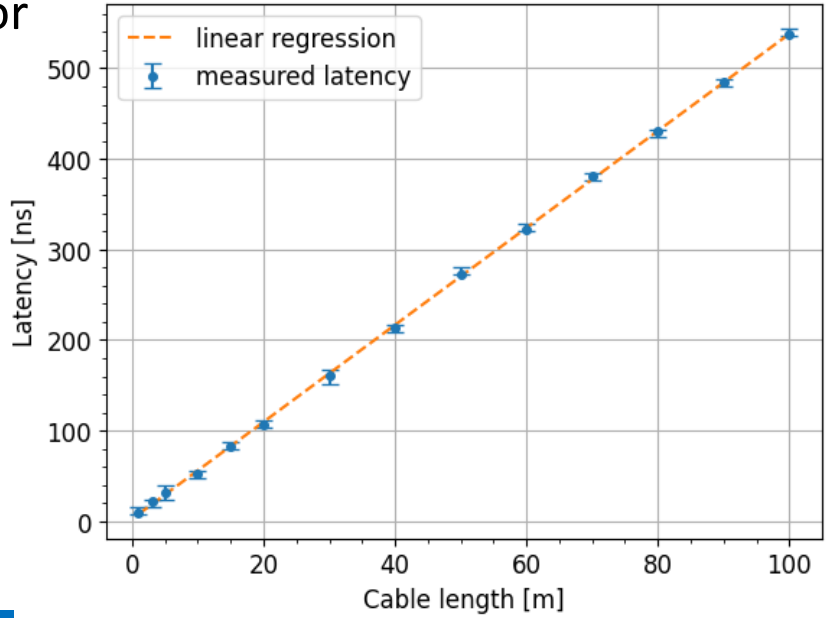
Evaluation: Timing accuracy

- We must first validate the timing accuracy of EFCC
- Measured latency of commercial products that we tested was unstable
  - Possibly synchronization issue between sender and receiver appliances

- We opted to measure the latencies of different lengths of CAT6A Ethernet cables (2 m to 100 m)
  - RJ45 couplers were used for 30 m or longer lengths
    - No latency impact was observed
- Compared them with theoretically expected latencies
  - Copper Ethernet cable Nominal Velocity of Propagation (NVP) is estimated at 65% to 70% of the speed of light in a vacuum according to the datasheet

Evaluation: Timing accuracy

- We set up one EFCC port as a frame generator and another port as a frame capture
- Send 1000 frames (1518 and 64 bytes)
- Observed a variation of ± 8ns
- Latency increased with the cable length
- 100m cable took approximately 500 ns
  - Signal propagation speed was 200,000,000 m/s
  - 66.7 % of the speed of light in a vacuum

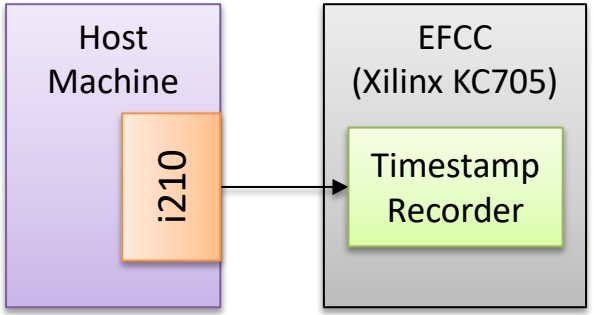


1000 frames (1518 bytes) average latency

Measured Latency matched theoretical value!

Evaluation: Frame capture capability

- For neutrality, we used a normal GbE NIC (e.g., Intel i210) attached to a Linux machine
- ***iperf*** sent frames exactly back-to-back
  - Frames of a fixed size at the line rate of GbE
  - Frame interval is constant and depends on the frame size

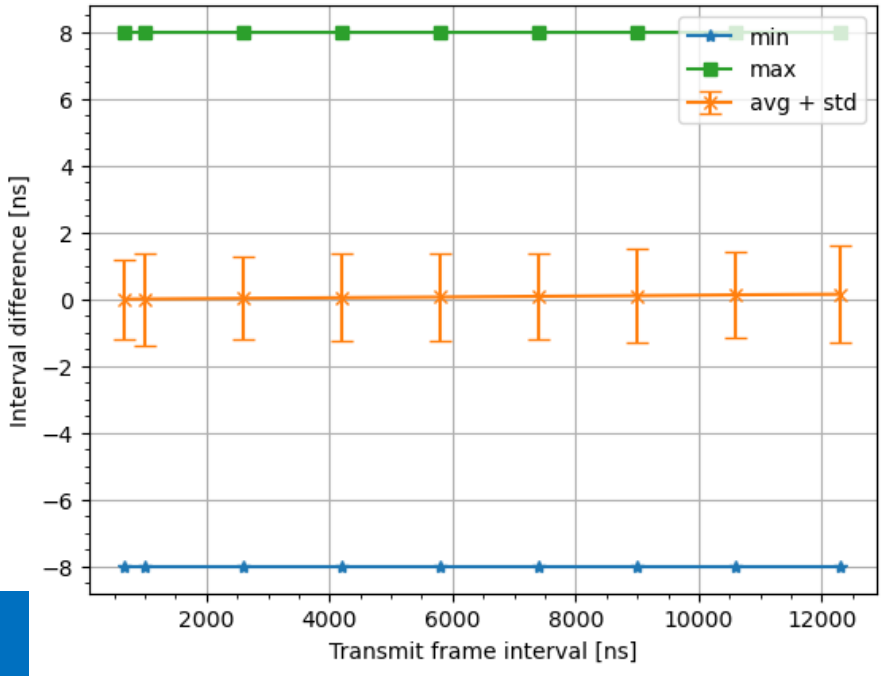


- Repeated the experiments with different frame sizes
- Compare the frame intervals recorded by the frame capture with the one generated by ***iperf***

Evaluation: Frame capture capability

- At 672 ns reference frame interval:
  - 98% of the captured frame intervals were exact
  - 2% were with ±8 ns
- At 12,304 ns reference frame interval:
  - 97% of the captured frame intervals were exact
  - 3% were with ±8 ns
- A possible jitter in the PHY IP of the FPGA can be the reason for ±8 ns deviation
  - Inevitable but acceptable

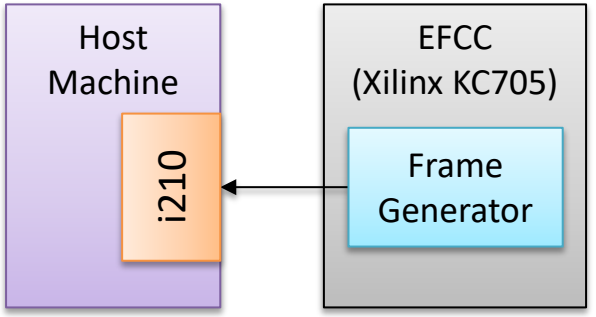
Accurate and Low Jitter Frame Capture!



Measured frame interval deviation

Evaluation: Frame generation capability

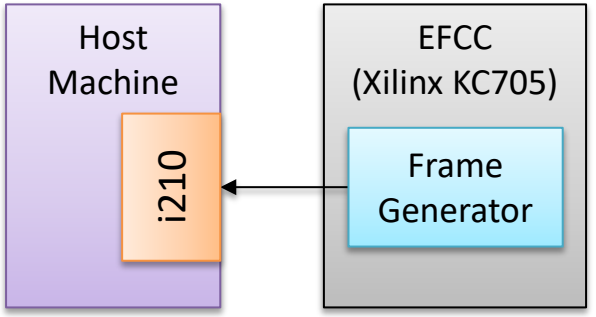
- For neutrality, we used Intel i210 NIC attached to a Linux machine
  - i210, which supports hardware-based receive timestamp, can record at an 8-ns granularity
- In EFCC, we defined a sequence of mixed frames to be transmitted
- The Frame Generator BRAM can host 8192 entries:
  - 4097 NOP frames
  - 4095 frames for transmission
- For frame capture on the host, ***tcpdump*** is used



Predefined but randomly chosen:
- ✓ *VLAN tag (0~4095)*
- ✓ *PCP code (0~7)*
- ✓ *MAC addresses (1~256)*
- ✓ *frame size (64~1522 bytes)*
- ✓ *Frame Interval (0 ~ 524,840 ns)*

Evaluation: Frame generation capability

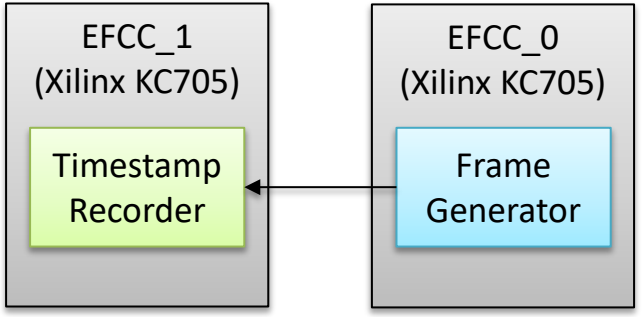
- No frame loss was observed
- Captured data matched the sent sequence
- The average of differences between specified and measured frame intervals was **3ns** with a standard deviation of **5ns**
  - 52% exact match
  - 40% 8ns larger than the predefined one
  - 1.7% 16ns larger than the predefined one
  - 6.3% 8ns smaller than the predefined one
- Possibility that the timestamp clock in the capturing network interface card was slightly slower than that of the Frame Generator



Predefined but randomly chosen:
- ✓ *VLAN tag (0~4095)*
- ✓ *PCP code (0~7)*
- ✓ *MAC addresses (1~256)*
- ✓ *frame size (64~1522 bytes)*
- ✓ *Frame Interval (0 ~ 524,840 ns)*

Evaluation: Frame generation capability

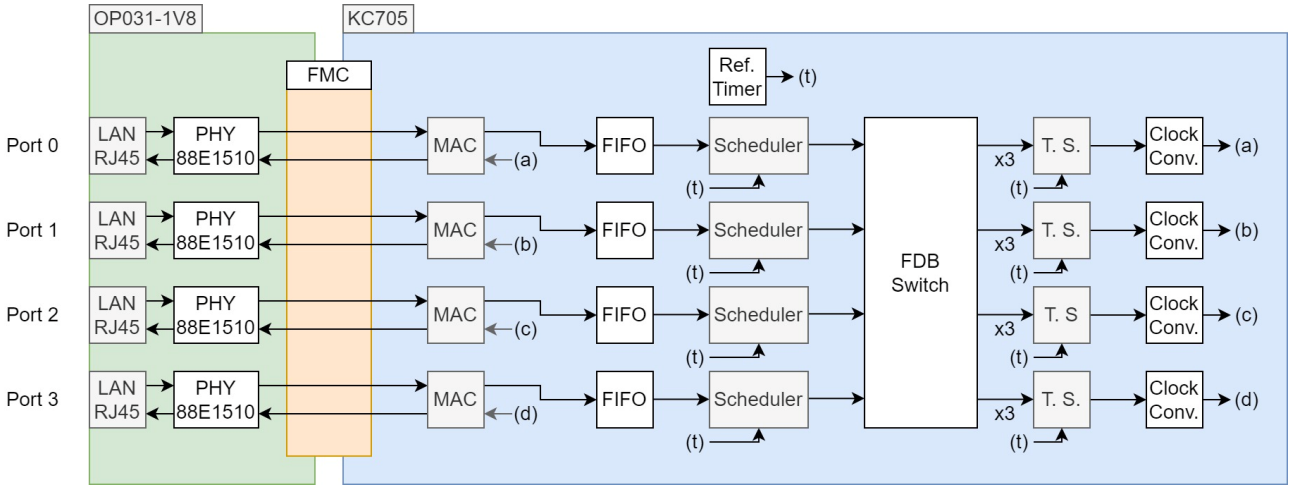
- For comparison, a Timestamp Recorder on another FPGA board was used instead of ***tcpdump***
- Same method and experiments are reproduced
- The average of differences between specified and measured frame intervals was **0ns**
  - 52% exact match
  - 24% 8ns larger than the predefined one
  - 24% 8ns smaller than the predefined one
- A possible jitter in the PHY IP of the FPGA can be the reason for ±8 ns deviation
  - Inevitable but acceptable



Predefined but randomly chosen:
- ✓ *VLAN tag (0~4095)*
- ✓ *PCP code (0~7)*
- ✓ *MAC addresses (1~256)*
- ✓ *frame size (64~1522 bytes)*
- ✓ *Frame Interval (0 ~ 524,840 ns)*

Accurate, Low Jitter, and Flexible Frame Generation!

Evaluation: Latency bound of multiple flows

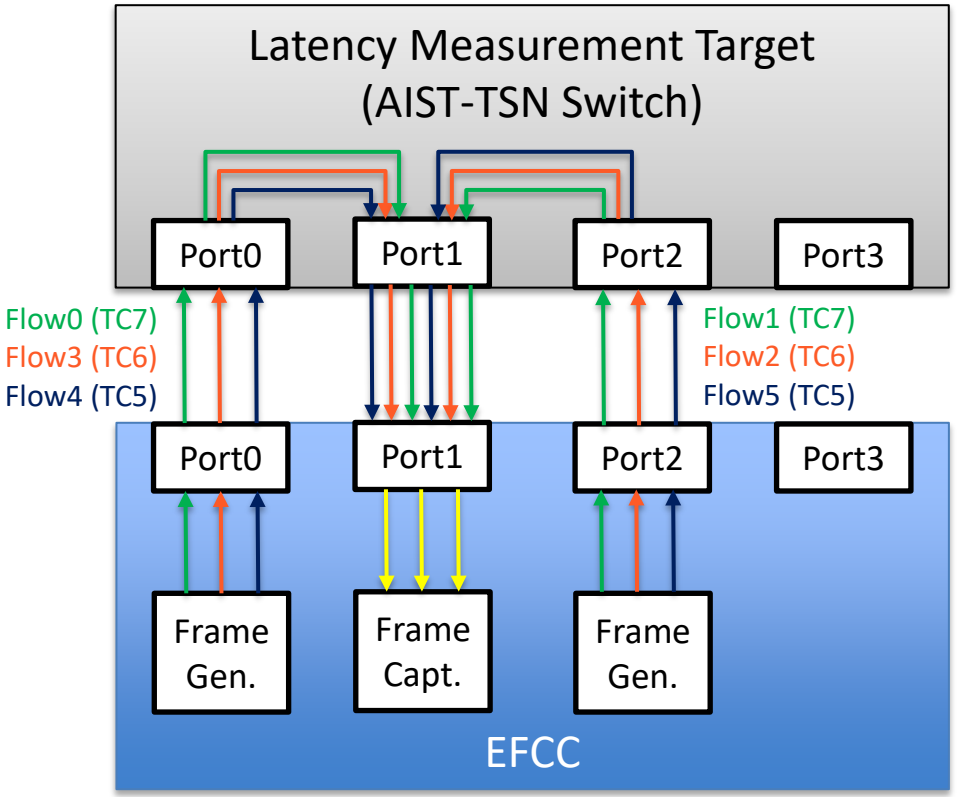


- *AIST-TSN Switch* [2] is the first open-source implementation of TSN-Switch supporting ATS
  - The evaluation of multiple flows with different characteristics contending with each other could not be done due to the absence of a reliable measurement mechanism dedicated to TSN research
  - It is the perfect case to show the usefulness of the proposed EFCC

[2] Hardware design and Evaluation of an FPGA-based Network Switch Supporting ATS for TSN, IEEE Access 2024

Evaluation: Latency bound of multiple flows

- Three traffic classes are considered :
  - TC7 ATS (Highest priority)
  - TC6 ATS (Mid priority)
  - TC5 Strict Priority (Lowest priority)
- For each traffic class, two flows were generated by EFCC and sent through Port0 and Port2
- All flows are transmitted to the AIST-TSN switch to contend for Port1
- We observe the measured contention latency (maximum - minimum latencies) and compare it to the theoretical model [2,3]



[2] Hardware design and Evaluation of an FPGA-based Network Switch Supporting ATS for TSN, IEEE Access 2024
[3] Urgency-based scheduler for time-sensitive switched ethernet networks, Euromicro 2018

Evaluation: Latency bound of multiple flows

Table1. Target flows parameters, measured latency, and theoretical contention latency bounds

		Flow 0	Flow 1	Flow 2	Flow 3	Flow 4	Flow 5
Transmitted flows' parameters	Traffic class	7 (ATS)	7 (ATS)	6 (ATS)	6 (ATS)	5 (SP)	5 (SP)
	Input port of the switch	0	2	2	0	0	2
	Frame size (byte)	1542	1542	1542	1542	1542	1542
	Transmit rate (Mbps)	100	100	100	100	800	800
	Burst size (byte)	3084	1542	3084	1542	13878	13878
Theoretical ATS contention delay bound (us)	$\max_{g \in F_s(k,f)} d_{CO,max}(k,g)$	37.008	37.008	92.520	92.520	N.A.	N.A.
Measured latency (us)	Averaged	59.014	58.436	63.697	61.934	315.508	316.185
	Minimum	51.648	51.640	51.640	51.640	100.064	13.736
	Maximum	76.216	74.808	99.384	100.064	379.416	379.152
	Maximum - Minimum	24.568	23.168	47.744	48.424	279.352	365.416

- No frame loss for ATS flows (Flow 0~3) and 60% frame loss for SP flows (Flow 4-5)
- ATS flows of the highest priority TC7 (Flow 0-1)
  - Contention delay were approximately 24.6 and 23.2 µs
  - Theoretical upper bound was calculated as 37.0 µs
- ATS flows of the mid priority TC6 (Flow 2-3)
  - Contention delay were approximately 47.7 and 48.4 µs
  - Theoretical upper bound was calculated as 92.5 µs

Evaluation: Hardware complexity

Table2. EFCC System hardware complexity

Component	MAC	FIFO	Frame Gen.	AXI4 Stream Switch	Timestamp Rec.	Clk. Conv.	Others	Total	Utiliz. (%)
LUTs	8299	2796	19148	594	22735	265	7853	61690	30.27
FFs	13492	1992	27959	1163	33400	448	12771	91225	22.38
LUTRAMs	1357	2048	3875	0	5152	32	2198	14662	22.91
BRAMs	0	0	152	0	256	0	2	410	92.25

- 4 Frame Generators + 8 Timestamp Recorder
- Reasonable portions of the LUTs, FFs, and LUTRAMs are consumed (22~30%)
- BRAM resources are reaching the limit (≃92%)
  - The Frame Generators Timestamp Recorder are the main consumers
    - Several buffers for the transmission information and the recorded timestamps are stored
  - KC705 is among the most affordable FPGA boards with limited resources
    - EFCC can be easily attached to a host machine if larger frame-generation/capture buffers are needed.
  - High-end FPGAs can be used for larger specifications or additional TSN measurement features

Outline

- TSN opportunities and challenges
- Motivation and solution
- EFCC architecture and implementation
- Evaluation environment and results
- Conclusion and Future work

Conclusion

- The proposed FPGA-based Ethernet Frame Crafter & Capture (EFCC) directly addresses the critical research gap in generating and measuring highly precise (ns-level) and mixed traffic flows for Time-Sensitive Networking.
  - Not limited to only TSN and can be used for various network studies!
- The system successfully handles concurrent and multi-flow traffic, crucial for real-world simulation, within the stipulated latency bounds.
- EFCC is portable and open-source allowing other researchers to verify and build upon the findings, making it an accessible tool for the academic community.

Future Work

- Using EFCC, validate TSN switches in more complex situations where multiple and various flows with different sizes and parameters coexist in TSN networks.
- Scale up to higher speeds (10GbE / 25GbE) to meet modern data center and 5G requirements.
  - Succeeded to develop 10G Switch and measurement tool!
  - Open-source repository to publicly published soon
- Address the optimization and configuration of TSN constraints for a more complete and comprehensive solution

Repository

- All the source code for EFCC as well as CBS and ATS switches, is available under a very permissive open-source license
  - MIT License
  - https://github.com/CCIRT/aist-tsn
- We provide Jupyter Notebooks examples to test EFCC
  - No need for expert knowledge about hardware design or FPGA technology
  - https://github.com/CCIRT/aist-tsn-efcc/tree/main/example
- For reproducible research, other artifacts are also available.

Thank you!!